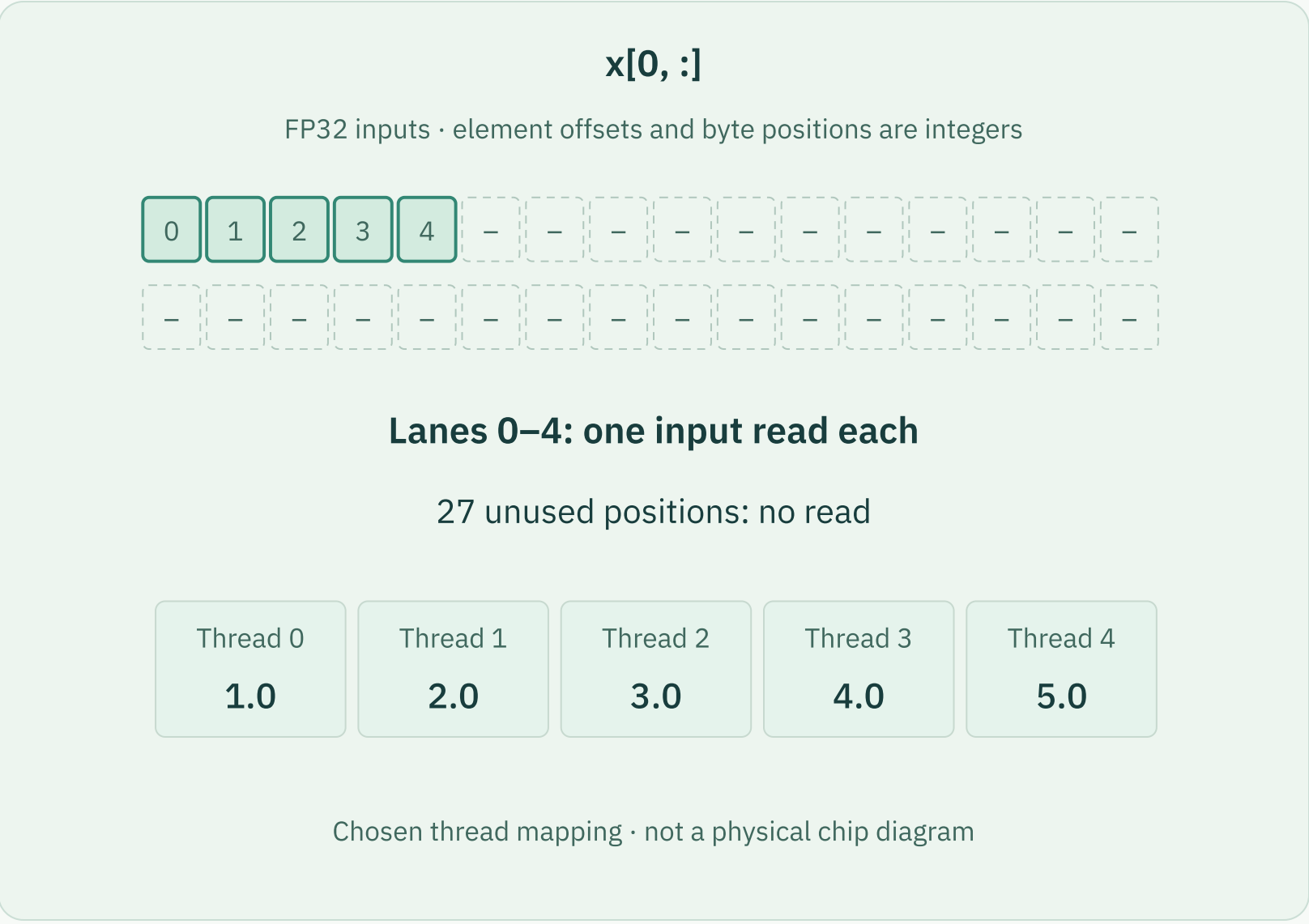
1 / 8

Five reads belong to one warp.

Follow one input read from each participating thread.

> All five threads belong to the same warp.



Copyright © 2026 Christopher Snow

2 / 8

Group bytes into sectors.

One aligned sector covers 32 bytes.

> Every named allocation starts at a 32-byte boundary.

One sector = 32 bytes

Eight FP32 positions · four bytes each

0–3	4–7	8–11	12–15	16–19	20–23	24–27	28–31
bytes	bytes	bytes	bytes	bytes	bytes	bytes	bytes

Sector 0: bytes 0–31

Sector 1: bytes 32–63

Sector 2: bytes 64–95

Region boundaries stay fixed when a view changes.

Copyright © 2026 Christopher Snow

3 / 8

A first row shares one sector.

Five values use 20 of the sector’s 32 bytes.

> Useful bytes count requested FP32 values.

x[0, :]

FP32 inputs · element offsets and byte positions are integers

Thread	Value	Element offset	Byte position	Sector
0	**1.0**	0	0	0
1	**2.0**	1	4	0
2	**3.0**	2	8	0
3	**4.0**	3	12	0
4	**5.0**	4	16	0

Sector 0

Bytes 0–31

Threads 0, 1, 2, 3, 4

20 useful bytes · 1 sector · 32 covered bytes

Aligned allocations · one modelled input read · no cache or timing model

Copyright © 2026 Christopher Snow

4 / 8

A column reaches three sectors.

Same useful byte count. More regions touched.

> A sector groups addresses, not tensor coordinates.

x[:, 0]

FP32 inputs · element offsets and byte positions are integers

Thread	Value	Element offset	Byte position	Sector
0	**1.0**	0	0	0
1	**6.0**	5	20	0
2	**11.0**	10	40	1
3	**16.0**	15	60	1
4	**21.0**	20	80	2

Sector 0	Sector 1	Sector 2
Bytes 0–31	Bytes 32–63	Bytes 64–95
Threads 0, 1	Threads 2, 3	Threads 4

20 useful bytes · 3 sectors · 96 covered bytes

Aligned allocations · one modelled input read · no cache or timing model

Copyright © 2026 Christopher Snow

5 / 8

Packing changes the input layout.

The same column values now occupy consecutive positions.

> Copy construction and output stores are excluded.

x[:, 0].contiguous()

FP32 inputs · element offsets and byte positions are integers

Thread	Value	Element offset	Byte position	Sector
0	**1.0**	0	0	0
1	**6.0**	1	4	0
2	**11.0**	2	8	0
3	**16.0**	3	12	0
4	**21.0**	4	16	0

Sector 0

Bytes 0–31

Threads 0, 1, 2, 3, 4

20 useful bytes · 1 sector · 32 covered bytes

Copyright © 2026 Christopher Snow

6 / 8

Neighbouring values can cross a boundary.

Row one begins partway through sector zero.

> The original row-major storage has not moved.

x[1, :]

FP32 inputs · element offsets and byte positions are integers

Thread	Value	Element offset	Byte position	Sector
0	**6.0**	5	20	0
1	**7.0**	6	24	0
2	**8.0**	7	28	0
3	**9.0**	8	32	1
4	**10.0**	9	36	1

Sector 0
Bytes 0–31
Threads 0, 1, 2

Sector 1
Bytes 32–63
Threads 3, 4

20 useful bytes · 2 sectors · 64 covered bytes

Copyright © 2026 Christopher Snow

7 / 8

Pause: where does row three land?

Five neighbours, starting at element offset 15.

> Pause before revealing the sector groups.

x[3, :]

FP32 inputs · element offsets and byte positions are integers

Thread	Value	Element offset	Byte position	Sector
0	**16.0**	15	60	?
1	**17.0**	16	64	?
2	**18.0**	17	68	?
3	**19.0**	18	72	?
4	**20.0**	19	76	?

Which sector contains each byte position?

Useful bytes: ? · sectors: ? · covered bytes: ?

Copyright © 2026 Christopher Snow

8 / 8

Count regions, then state the scope.

The boundary falls after the first selected value.

> Sector coverage is not a measurement of DRAM traffic.

x[3, :]

FP32 inputs · element offsets and byte positions are integers

Thread	Value	Element offset	Byte position	Sector
0	**16.0**	15	60	1
1	**17.0**	16	64	2
2	**18.0**	17	68	2
3	**19.0**	18	72	2
4	**20.0**	19	76	2

Sector 1
Bytes 32–63
Threads 0

Sector 2
Bytes 64–95
Threads 1, 2, 3, 4

20 useful bytes · 2 sectors · 64 covered bytes

Copyright © 2026 Christopher Snow